0. Source Basis

- lectures/current-2026/ANLP Session1_Week1.pdf — authoritative 2026 slide deck
- notebooks/current-2026/session-01/ANLP_Session_1_NLP_Basics_Part_1.ipynb
- notebooks/current-2026/session-01/ANLP_Session_1_NLP_Basics_Part_2.ipynb
- notebooks/current-2026/session-01/ANLP_Session_1_HW.ipynb
- lectures/raw/archive-2025/ANLP Session1_Week1_After Session-2.pdf — comparison source

The 2026 slide deck is authoritative for staff, attendance, assessments, academic integrity, the subject roadmap, and current examples. The 2025 deck and inherited notebooks remain useful supporting sources where they align with the current lecture.

0.4 Practical Notebook Map

Part 1 follows the basic NLP pipeline from text acquisition through tokenisation, stopword handling, stemming, lemmatisation, POS tagging, named-entity chunking, dependency parsing, and regular expressions. It uses both **NLTK** and **spaCy** , making their different representations visible.

Part 2 applies preprocessing to a CSV corpus with **pandas** , then uses frequency distributions, n-grams, plots, and word clouds for exploratory text analysis. These outputs require interpretation: preprocessing choices change which patterns become visible, and frequency alone does not establish meaning or importance.

The original homework asks students to implement input collection, basic spaCy analysis, a custom tokenizer, citation and URL regexes, corpus-frequency comparisons, and web scraping. A separate worked learning copy is available at notebooks/current-2026/session-01/ANLP_Session_1_HW_Worked.ipynb; the source exercise remains unchanged.

0.1 Teaching and Participation in 2026

Dr Arnick Abdollahi is the subject coordinator and teaches alongside **Sarah Fawcett** and **Mutaz Abu Ghazaleh** . General questions should be shared through Canvas so the cohort can benefit, content questions should go to the corresponding lecturer, and extension requests should go to the coordinator.

Face-to-face attendance is mandatory and recordings are not provided. Students should log **Qwickly attendance** when requested; absence from more than three sessions may affect the AT3 mark. Canvas is authoritative for announcements, subject requirements, extension requests, assessment briefs, and the current schedule.

0.2 Assessment Structure

1. **AT1 — NLP for Data Analysis:** individual Python notebook and report, worth 30%, due **19 August 2026** .
2. **AT2A — Conference Poster:** group digital poster, worth 10%, due **23 September 2026** .
3. **AT2B — Application/Site:** group application and code with peer-assessment contribution, worth 40%, due **14 October 2026** .
4. **In-class project presentation:** group presentation on **Monday 19 October, 5:30–8:30 pm** .
5. **AT3 — Critical and Ethical Reflection:** individual task, worth 20%, due **28 October 2026** .

AT1 uses parliamentary submissions concerning skilled migration. The task requires exploration, manipulation, visualisation, interpretation, and a short interpretive summary that answers the **“so what?”** question. Students submit both the .ipynb notebook and a report exported to PDF.

0.3 Generative AI and Academic Integrity

Generative AI may be used as a study aid for explanations, starter ideas, language correction, debugging guidance, and research assistance. Outputs must be validated, copied or reworked material must be declared, and submissions must remain original. The 2026 slides explicitly warn that copy-pasting generic AI output is not interpretation: students must expand acronyms, explain code and decisions, connect evidence, and write in their own analytical voice.

1. What Applied NLP Is

1.1 Natural Language Processing

Natural Language Processing (NLP) is an interdisciplinary area connecting **artificial intelligence** , **computer science** , **linguistics** , and **human language** . It develops computational methods that help computers process, interpret, and manipulate language.

Human language has enormous analytical value, but it is harder to process than numerical data. A computer receives encoded symbols rather than human understanding, so an NLP system must build useful representations of linguistic structure and context.

1.2 Text Mining

Text mining , also called text analysis or text analytics, applies NLP methods to extract useful insights from unstructured text. Applied NLP connects linguistic analysis with practical questions, computational tools, and the communication of defensible findings.

2. Why Language Understanding Is Difficult

2.1 Missing Context and World Knowledge

Language rarely states every piece of background knowledge explicitly. For example, web descriptions may associate pizza with the adjectives “frozen,” “free,” and “cold,” even though people normally expect to buy freshly made, hot pizza. A model that learns only from observed co-occurrence can therefore miss ordinary human expectations.

2.2 Language Complexity

The slides illustrate several additional difficulties:

- **Subtlety:** a negative perfume review can be phrased indirectly as advice to wear the fragrance only at home with the windows closed.
- **Thwarted expectations and ordering effects:** a review may begin with positive evidence and reverse its judgement only after “however.”
- **Sarcasm:** “Great idea, now try again with a real product development team” expresses criticism despite positive-looking words.
- **Implicit knowledge:** “Please take a seat” conventionally means to sit down, not to carry a chair away.

These examples show why counting positive and negative words is insufficient: meaning depends on order, context, shared knowledge, and communicative intent.

3. Historical and Linguistic Foundations

3.1 From Symbolic to Data-Driven NLP

The history presented in the slides moves from early machine-translation ambitions, through symbolic and logic-based systems such as **Prolog** and **SHRDLU** , toward empirical, probabilistic, corpus-based, and modern neural methods. The practical lesson is not that newer methods always replace older ones. Classical methods can be cheaper, easier to deploy, and nearly as effective for some industry problems.

3.2 Levels of Language

NLP operates at several connected levels:

- **Morphology** examines word formation and relationships between word forms.
- A **lexicon** is the vocabulary or dictionary of a language.
- **Syntax** describes grammatical structure and relationships among words.
- **Semantics** concerns the meaning of words, sentences, or complete texts.
- **Discourse** concerns coherence and meaning across multiple sentences.
- **Pragmatics** uses situation and world knowledge to infer intended meaning.

The word “bark” demonstrates semantic ambiguity: it can describe a dog’s action or the outer layer of a tree. Similarly, “interest” can refer to curiosity or a financial charge. POS and syntactic information help later systems determine the intended sense.

4. Tokenisation

4.1 Token Types

Tokenisation divides text into units that an NLP system can process. The unit depends on the task:

- **Word tokens:** “The cat says meow” becomes four word units.
- **Character tokens:** “Hello” becomes H, e, l, l, o.
- **Subword tokens:** “unhappiness” may become un + happy + ness.
- **Sentence tokens:** a document is segmented into sentences for tasks such as translation, summarisation, or sentence-level sentiment analysis.

The 2026 slides frame tokenisation as the first stage of **digitising text** . Computers require digitally encoded text and numerical references; token boundaries determine the units that later become positions, counts, tags, or vectors.

4.2 Tokenisation Challenges

Tokenisation is not simply splitting on spaces. A tokenizer needs a policy for punctuation, contractions such as “don’t,” compound or hyphenated expressions, emojis, hashtags, URLs, and languages such as Chinese that do not place spaces between every word. Different token boundaries preserve different information, so there is no universally correct tokenizer.

In Part 1, the notebook makes this distinction concrete:

```
from nltk.tokenize import sent_tokenize, word_tokenize
sentences = sent_tokenize(mytext)
tokens = word_tokenize(mytext)
```

The homework then asks students to compare a custom regular-expression tokenizer with word_tokenize, especially for contractions, punctuation, hyphenated words, and URLs.

Deep Dive — A Token Is a Modelling Choice

A token is not a natural unit with one universally correct boundary. A sentiment system may preserve n't because it carries negation, a search system may keep a URL intact, and a language model may split an unfamiliar word into reusable subwords. Tokenisation therefore defines the vocabulary and affects every downstream count, feature, and prediction.

5. Syntax, POS Tagging, and Dependency Structure

5.1 Syntax and Dependency Structure

Syntax matters because word order and grammatical relationships affect meaning: “He ate the fish” is not equivalent to “The fish ate him.” A **dependency tree** represents grammatical relationships, normally treating the main verb as the root and attaching subjects, objects, determiners, and other dependents.

5.2 Part-of-Speech Tagging

Part-of-speech (POS) tagging assigns a lexical category to each token. The slide example produces tags such as:

```
Jim/NNP bought/VBD 300/CD shares/NNS of/IN
Acme/NNP Corp/NNP in/IN 2006/CD
```

Here, NNP means singular proper noun, VBD means past-tense verb, CD means cardinal number, and NNS means plural noun. Part 1 performs tagging with nltk.pos_tag(tokens) and then builds a chunk/dependency-style tree.

POS information supports lemmatisation, word-sense disambiguation, named entity recognition, information extraction, parsing, speech systems, authorship attribution, and machine translation.

Deep Dive — Linguistic Annotations Are Predictions

POS tags, dependency relations, and named entities are model outputs rather than unquestionable facts. Their accuracy can change with language, domain, spelling, and sentence structure. Inspect errors before treating annotations as reliable features, especially for informal, specialised, or multilingual text.

6. Named Entities, Meaning, and Reference

6.1 Named Entity Recognition

Named Entity Recognition (NER) identifies and categorises mentions such as people, organisations, locations, quantities, and times. In the slide example, “Jim” is a person, “Acme Corp.” is an organisation, and “2006” is a time expression. The Part 1 notebook processes text with spaCy, extracts entities, and visualises them with displacy.

6.2 Ambiguity and Reference

Word-sense disambiguation selects the intended meaning of an ambiguous word from context. **Anaphora or co-reference resolution** determines which phrases refer to the same entity. In “John put the carrot on the plate and ate it,” a system must infer what “it” refers to; difficult cases require discourse and world knowledge rather than local word matching alone.

7. Regular Expressions

A **regular expression** matches character patterns and is useful for extracting structured information from text, including email components, titles, subject codes, addresses, citations, and URLs. The Part 1 notebook introduces anchors, character classes, quantifiers, and word boundaries.

For example, this pattern finds complete words beginning with either uppercase or lowercase C:

```
re.findall(r"\b([Cc]\w+)\b", test_string)
```

The boundary \b prevents the match from beginning in the middle of a larger word, [Cc] handles both cases, and \w+ consumes one or more word characters. The homework extends this work by extracting author-year citations and URLs.

> ### Deep Dive — Regex Is Precise but Brittle
>
> Regex is excellent when the target format is explicit and stable, but it does not understand meaning. A citation or URL pattern can fail on an unseen format or match text that only looks valid. Test patterns on positive examples, negative examples, and edge cases rather than judging them from one successful match.

8. From Raw Text to Basic Analysis

8.1 Collecting and Inspecting Text

Part 1 introduces three input routes: define text in a Python variable, scrape text from a web page with requests and **BeautifulSoup** , or read a file. The web-scraping examples also show an important limitation: code tied to a specific page structure can fail when that HTML structure changes.

8.2 Corpus Analysis Workflow

Part 2 moves from a single text to a CSV corpus of news articles:

1. Read the CSV into a pandas DataFrame.
2. Inspect it with info(), describe(), head(), and sample().
3. Calculate article word counts and study their distribution.
4. Join article text, tokenise it, and calculate raw frequencies with nltk.FreqDist.
5. Visualise frequent terms, including with a word cloud.
6. Lowercase the text and remove selected stopwords and punctuation.
7. Recalculate frequencies and interpret how the result changes.

Counts become meaningful only when connected to a question. Article length might support a question about changes in writing practices, while frequency patterns might reveal recurring people, places, institutions, or political topics.

Deep Dive — From Counts to Comparable Measures

The notebook begins with absolute word counts, but raw counts are not always comparable. A long document normally contains more occurrences of every term than a short document. Relative term frequency divides a term count by the document's total token count. The type–token ratio compares the number of unique token types with the total number of tokens and gives a simple view of lexical diversity, although it normally decreases as document length grows. Always state whether a "word" means a whitespace-separated string or a token produced by a particular tokenizer.

$$N_d = \sum_{i=1}^{|d|} 1 = |d|$$

Document length: the number of tokens in document d.

$$\mathrm{TF}(t,d) = f_{t,d}$$

Absolute term frequency: the number of occurrences of term t in document d.

$$\text{RelativeTF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Relative frequency controls for documents with different lengths.

$$\text{TTR}(d) = \frac{|V_d|}{N_d}$$

Type–token ratio: unique token types divided by all tokens; it is length-sensitive.

8.3 N-Grams and Stemming

The 2026 deck extends basic descriptive analysis beyond single-token counts. An **n-gram** is a contiguous sequence of tokens: a **bigram** contains two, and a **trigram** contains three. N-grams preserve a small amount of local order and can reveal recurring phrases that unigram frequency misses.

Stemming groups related surface forms by reducing them to a common stem-like representation. This can consolidate counts, but the output may not be a valid dictionary word and unrelated words can sometimes be conflated. As with stopword removal, stemming should be justified by the analytical task.

9. Cleaning Is Task-Dependent

9.1 Stopwords, Punctuation, and Information Loss

Stopwords are frequent function words that often contribute little to a particular frequency analysis. Libraries provide standard lists, but the notebook also adds custom stopwords relevant to its corpus. Removing them can make content-bearing terms more visible.

However, cleaning is not automatically beneficial. Punctuation can convey emotion or sentence boundaries, and stopwords such as “not” can reverse sentiment. Removing apostrophes turns “it’s” into “its” and “I’m” into “im,” which may create ambiguity. Therefore, preprocessing should be iterative: inspect the data, apply a justified transformation, compare the output, and record both the benefit and information loss.

Deep Dive — Keep an Auditable Cleaning Pipeline

Retain the original text and create transformed versions in new variables or columns. Record the order of operations, custom stopwords, tokenizer, library versions, and any rows removed. This makes the analysis reproducible and lets you trace a surprising result back to a specific preprocessing decision.

10. Notebook Study Summary

10.1 Part 1 — From Raw Text to Linguistic Structure

Part 1 begins with three ways to obtain text: a Python variable, interactive input, and HTML retrieved with **requests** and parsed with **BeautifulSoup** . It then demonstrates how the same text can be represented at progressively richer levels:

1. sentence and word tokens establish processing units;
2. stopword filtering, stemming, and lemmatisation normalise selected forms;
3. POS tags attach grammatical categories;
4. named entities identify spans such as people and organisations;
5. dependency relations represent head–dependent structure; and
6. regex extracts explicitly formatted patterns.

The central lesson is that every representation answers a different question. A stemmer is useful for consolidating counts but does not analyse grammar; a dependency parse adds syntax but does not guarantee correct interpretation; and regex is precise only when the target format is stable.

10.2 Part 2 — From a Corpus to Exploratory Evidence

Part 2 shifts the unit of analysis from one string to a collection of documents in a **pandas DataFrame** . The notebook models a repeatable EDA sequence: load, inspect, quantify missingness and length, tokenise, count, visualise, clean, and compare the result after cleaning.

The frequency table, bar chart, n-gram list, and word cloud are alternative views of the same corpus rather than independent proof. A good explanation states the denominator, identifies the preprocessing version used, interprets the visible pattern, and acknowledges that frequent language is not automatically important, positive, or representative.

10.3 Worked Homework — Testing the Building Blocks

The worked homework turns the lecture concepts into six small, testable tasks:

1. collect text while retaining a reproducible default;
2. inspect tokens and sentences with spaCy;
3. compare a transparent custom tokenizer with NLTK;
4. extract citations and URLs with regex;
5. compare corpus frequencies with standard, custom, and no stopword removal;
6. scrape the four MDSI core subjects from the current UTS course page.

The tokenizer comparison exposes a practical trade-off. The custom rules split “state-of-the-art,” preserve a complete URL, and convert “don’t” to do + n't, while NLTK preserves the hyphenated compound and splits the URL. Neither behaviour is universally superior; the appropriate boundary policy depends on the downstream task.

The corpus exercise also shows why results must be compared across preprocessing choices. Without stopword removal, grammatical function words dominate. Standard stopwords reveal more content words, custom stopwords remove corpus-specific high-frequency names, and retaining punctuation makes marks such as commas and periods among the most frequent tokens.

10.4 End-to-End Practical Workflow

```
collect text
→ inspect its structure and quality
→ segment into sentences and tokens
→ add linguistic structure with POS, dependencies, and NER
→ extract targeted patterns with regex
→ calculate lengths and frequencies
→ inspect n-grams and stems when the question requires them
→ clean only what the task requires
→ visualise and interpret the result
```

The final step is interpretation. A chart or word cloud is not the conclusion; students should explain what the result means, how preprocessing affected it, what evidence supports the interpretation, and what limitations remain.

11. Learning Objectives

11.1 Current Subject Learning Outcomes

The 2026 subject learning outcomes require students to:

1. Understand core NLP and computational-linguistics concepts and limitations.
2. Apply text-mining techniques to unstructured data using advanced packages.
3. Evaluate complex problems and build practical NLP and LLM applications.
4. Interpret, extract value from, and communicate text-analysis insights.
5. Create audience-appropriate AI applications that address business problems.
6. Articulate NLP assumptions, strengths, weaknesses, ethical debates, and responsible AI practices.

11.2 Session 1 Learning Objectives

After Session 1, students should be able to:

1. Explain why human language is difficult for computers to interpret.
2. Distinguish morphology, syntax, semantics, discourse, and pragmatics.
3. Compare word, character, subword, and sentence tokenisation.
4. Explain how POS tagging, dependency structure, NER, and co-reference add linguistic information to raw text.
5. Use NLTK or spaCy for basic tokenisation, tagging, and entity extraction.
6. Use regular expressions to extract targeted patterns.
7. Inspect a text corpus and calculate word counts, frequencies, and n-grams.
8. Justify stopword and punctuation decisions by comparing outputs before and after cleaning.
9. Turn basic text-analysis results into a defensible interpretation.